PROGRAMMING & CS FUNDAMENTALS

Recursion Visual Guide

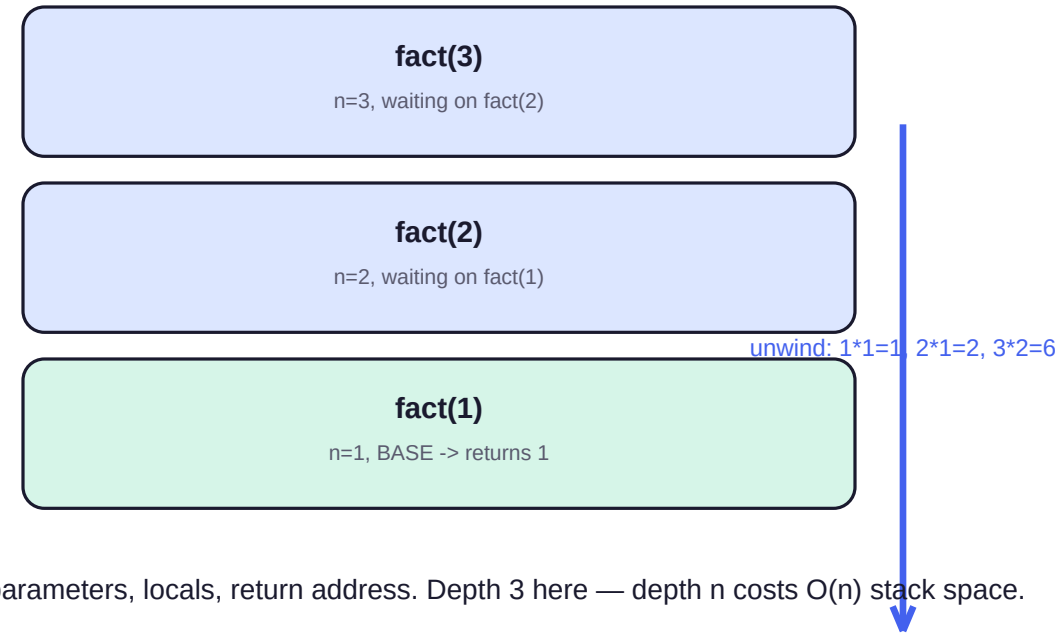
Functions calling functions —
the call stack, base cases, and when to iterate instead.

ANIMATED WALKTHROUGH
Step through pages to play the animation

STEP 1/6

The Call Stack: Frames in Action

Each call pushes a frame; return pops it. factorial(3) traced:



Frame holds: parameters, locals, return address. Depth 3 here — depth n costs O(n) stack space.

Python default recursion limit: 1000 frames. Deep recursion (long chains, big grids) needs iteration or sys.setrecursionlimit + care.

STEP 2/6

Anatomy: Base Case + Recursive Step

Two parts, both mandatory — miss one and it never ends.

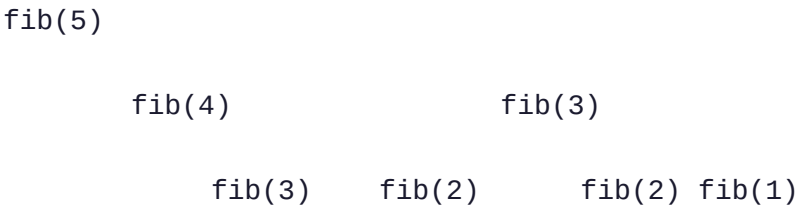
```
def factorial(n):
    if n <= 1: return 1          # BASE CASE: smallest input, no recursion
    return n * factorial(n - 1)  # RECURSIVE STEP: smaller problem

# every call must move TOWARD the base case: n -> n-1 -> ... -> 1
```

- Base case: the answer you know directly — without it: infinite recursion -> RecursionError.
- Progress: each call shrinks the problem — n-1, half the array, one subtree.
- Trust the recursion: assume factorial(n-1) works; your only job is the n-case.
- Multiple base cases are fine (fib needs n under 2); zero base cases is a bug.

STEP 3/6

Trace It: Fibonacci's Explosion

Beautiful definition, catastrophic runtime — without memoization.



~2^n calls. fib(50) = 12.5 TRILLION calls — heat death first.

```
# recurrence: T(n) = T(n-1) + T(n-2) + O(1)  =>  T(n) = O(2^n)
# fix: memoize (DP!) or iterate — both O(n)
```

Recursion + overlapping subproblems = exponential. The fix is memoization — which is exactly dynamic programming (see the DP guide).

STEP 4/6

Tail Recursion

Recursive call is the LAST action — in theory, optimizable to a loop.

```
# TAIL recursive: nothing left to do after the call
def fact_tr(n, acc=1):
    if n <= 1: return acc
    return fact_tr(n - 1, acc * n)   # last action = the call

# NOT tail: work remains after the call returns
def fact(n):
    if n <= 1: return 1
    return n * fact(n - 1)           # multiply AFTER return
```

- Tail-call optimization (TCO): reuse the frame instead of pushing — O(1) stack.
- Scheme/Haskell do TCO; Python and Java do NOT — write the loop yourself there.
- Accumulator parameter carries the 'work so far' — the standard tail-recursion transform.

STEP 5/6

Recursion vs Iteration

Same power, different trade-offs.

Readability

recursion: mirrors problem structure | iteration: loop bookkeeping visible

Tree/graph walks

recursion: natural (DFS) | iteration: explicit stack needed

Overhead

recursion: call frames + returns | iteration: minimal

Depth limits

recursion: stack overflow risk | iteration: no limit

State

recursion: implicit in frames | iteration: explicit variables

Rule: recurse for branching structures (trees, backtracking); iterate for long linear chains. Convert with an explicit stack when both matter.

STEP 6/6

Recursion Patterns

Five shapes you'll meet again and again.

Linear

fact(n), linked-list walk — one recursive call

Binary

fib, merge sort — two calls, combine results

Multi-way

tree DFS, permutations — loop over children

Backtracking

try, recurse, UNDO — N-queens, Sudoku

Divide & conquer

split, solve, merge — quicksort, mergesort

STEP ✓I✓

Key Takeaways

Recursion — Remember

- Each call pushes a frame (params, locals, return addr); depth n costs O(n) stack.
- Base case + progress toward it: miss either and you get infinite recursion.
- Naive fib is O(2^n) from overlapping subproblems — memoize or iterate to O(n).
- Tail recursion (call is the last action) enables TCO — but Python/Java don't do it.
- Recurse for branching structures, iterate for long chains; convert with an explicit stack.

Keep exploring

Pair this guide with the matching video walkthroughs and hands-on labs at interstitiumlabs.dev